

Computational Physics and Modelling – Assessment 2

N-Body Problem

Elijah Gill

12 January 2025

1 Code Description

This section of the report breaks down the functions created for this assessment. The functions can be found in the file: “NBody_Functions.py” and is imported at the start of the Jupyter notebook. The Jupyter notebook, named “Nbody_Problem_Notebook.ipynb” contains the code used for code verification, convergence and a few comparisons with online example systems. The python file starts with importing numpy for its array functionality, matplotlib for graph plotting and pandas for data file reading. Following that are some useful global constants, such as the gravitational constant G expressed in different units, which are shown in the respective comments.

1.1 Body() Class

Firstly, a class called “Body” is defined. This class represents a specific body that will be used in a simulation, it contains its position, velocity and mass and has the following method functions:

- `Body.values()` returns a list containing three elements; the body’s position, velocity and mass.
- `Body.KE()` returns the kinetic energy of the body using, $KE = \frac{1}{2}mv^2$. Where m is the body’s mass and v is the body’s velocity.
- `Body.GPE(bodies)` returns the gravitational potential energy on the body due to all other bodies in the system, which is given as a list of `Body()` objects. It is calculated by summing, $GPE = \frac{GMm}{r}$, for each mass in the system, where G is the gravitational constant, M is the mass of the body, m is the mass of another body in the system and r is the distance between the body and another body in the system.

When creating a system, the user generates as many bodies as needed using this class, giving each body a unique position, velocity and mass. All the bodies are then appended to a list which is what is passed to all relevant functions. It was helpful to create the following functions to split the bodies’ data into separate parts for easier processing:

- `find_positions()` takes the bodies list and returns a list of each body’s position in the system.
- `find_velocities()` takes the bodies list and returns a list of each body’s velocity in the system.
- `find_masses()` takes the bodies list and returns a list of each body’s mass in the system.

1.2 Force and Energy Functions

This section of the python file contains functions relevant to calculating physical quantities of the system. It contains the following functions:

- `force_gravity()` finds the force due to gravity between two bodies using the equation,

$$\vec{F} = -\frac{Gm_1m_2}{|\vec{r}|^3 + \epsilon^2}\vec{r} \quad (1)$$

where $\vec{F}$ is the force vector, G is the gravitational constant, m_1 , m_2 are the masses of the first and second objects respectively. $\vec{r}$ is the vector displacement between the two bodies and ϵ is a constant used to avoid division by zero and high force values when the displacement is very small.

- `system_energy()` finds the total energy of an N-Body system. It does this by summing the kinetic energies of each body calculated using `Body.KE()` and then looping through all the interactions between separate bodies to find the gravitational potential energies and summing them together. The final energy is the total kinetic energy minus the gravitational potential energy.
- `system_ang_momentum()` returns the total angular momentum of an N-Body system. It is calculated using this equation,

$$\sum_i^N m_i [\mathbf{r}_i \times \mathbf{v}_i + \mathbf{r}_i \times \mathbf{V} + \mathbf{R} \times \mathbf{v}_i + \mathbf{R} \times \mathbf{V}] \quad (2)$$

where m_i , $\mathbf{r}_i$, $\mathbf{v}_i$ is the mass, position and velocity of the i th body respectively. $\mathbf{R}$ is the position of the centre of mass and $\mathbf{V}$ is the velocity of the centre of mass. All the cross product calculations take advantage of numpy's vectorisation to do all the calculations without requiring a for loop, vastly speeding up one of the slowest functions in the code.

- `accelerations()` uses the `force_gravity()` function to cycle through each unique interaction between two bodies in the system and uses the force between the two bodies to calculate the accelerations due to this, the acceleration from each interaction is summed to find the total acceleration for each body. A list containing this information is returned.
- `find_semimajor_axis()` finds the average semimajor axis for an elliptical 2-body system. It finds the number of orbits using a provided time period to find the minimum and maximum x-values in this time range and calculating the semimajor axis using this. This function only gives a correct value if the semimajor axis of the system is along the x-axis.
- `find_period()` finds the average orbital time period for an elliptical 2-body system. It works based on the fact that in an elliptical orbit a body's x-velocity will change sign twice per orbit. The function checks when the second sign change happens and finds the timestep it happened on, then finds the time difference since the last orbit to find the time period.
- `check_kepler()` returns both the lefthand and righthand side of the equation for Kepler's third law so that they can be compared. The equation is,

$$\frac{P^2}{(a_1 + a_2)^3} = \frac{4\pi^2}{G(m_1 + m_2)} \quad (3)$$

where P is the time period, a_1 and a_2 are the semimajor axis for bodies 1 and 2 respectively, G is the gravitational constant, m_1 and m_2 are the masses for bodies 1 and 2 respectively.

1.3 Verlet Functions

This section contains two functions, the first is `verlet_step()` which computes a single step using the velocity verlet method. A step of the velocity verlet method is split into three equations,

$$v_{i+1/2} = v_i + \frac{1}{2}\delta a(x_i) \quad (4a)$$

$$x_{i+1} = x_i + \delta v_{i+1/2} \quad (4b)$$

$$v_{i+1} = v_{i+1/2} + \frac{1}{2}\delta a(x_{i+1}) \quad (4c)$$

where x_i , v_i are the initial position and velocity respectively, $v_{i+1/2}$ is the velocity half step and x_{i+1} , v_{i+1} are the full step position and velocity. δ is the timestep and $a(x_i)$ is the acceleration due to position. The function itself calls the `accelerations()` function at the start of the step, calculates the velocity half step and position full step for each body in the system, then uses the new positions to calculate the new accelerations and finally the velocity full step for all bodies. The positions and velocities are updated by directly changing the `Body()` class parameters.

The second function is `verlet_model()` and it computes the velocity verlet model over a specified time period with a specified timestep. It creates a copy of the system data to iterate over so that the original variables aren't directly changed and can be used elsewhere, and it returns a set of 5 lists

containing the time, positions, velocities, energy and angular momentum for every timestep of the simulation.

1.4 Plotting

The next section of the python file is for plotting functions and contains the following:

- `plot_orbits()` is the function that is used to visualise the orbit paths in a system as well as view the percentage change in energy and change in angular momentum each step. It works with any number of bodies but only has a unique colour combination for 9 bodies, this was chosen as the largest N-Body system presented has 9 bodies in total. The function makes use of Matplotlib's `gridspec`, which allows for more customisation on the plots generated. The function also has the capability to zoom in on the centre of the plot and save plots as png files.

1.5 Import Related Functions

With the python files, a text file named "Initial_Conditions.txt" is included in the submission, this contains a comma delimited table with the initial parameters for all the external 3-body problems to check the code against [1]. The function `verlet_from_table()` reads this text file and takes a system name to run the model and plot the results for each case, making running the code very user friendly.

2 Code Validation

The code in this project was validated using a 2-body system with elliptical orbits to verify that Kepler's laws were upheld. Kepler's first law states that orbits are ellipses which can be visually seen on the plots. The second law is that a planet sweeps out equal areas of its orbit in equal time intervals, this is due to the conservation of angular momentum, which can also be checked using the plots generated. Kepler's third law, however, is the relation between the orbital period and semimajor axis given by eq(3), this relation was checked by calculating the time period, P , and semimajor axis, a , separately and seeing if eq(3) was correctly satisfied.

All 3 laws were checked using five different systems each with 2 bodies, the following table shows the parameters used for each case.

Position 1	Position 2	Velocity 1	Velocity 2	Mass 1	Mass 2	Timestep	Time
[1, 0, 0]	[-1, 0, 0]	[0, 0.6, 0]	[0, -0.6, 0]	1	1	0.01	90
[1, 0, 0]	[-1, 0, 0]	[0, 1.0, 0]	[0, -0.5, 0]	1	2	0.01	90
[1, 0, 0]	[0, 0, 0]	[0, 100, 0]	[0, 0, 0]	1	10,000	0.0001	1
[1, 0, 0]	[0.2, 0, 0]	[0, 151.66, 0]	[0, 0, 0]	1	10,000	0.0001	1
[1, 0, 0]	[0, 0, 0]	[0, 7.071, 0]	[0, 0, 0]	0.1	50	0.0001	5

Table 1: Shows the initial conditions for each of the five cases used to check functionality with Kepler's laws.

After modelling each of the orbits, the values for the orbital time periods and semimajor axes could be found and used in the function `check_kepler()` to ensure Kepler's third law was maintained during the iterations.

Time Period, P	a_1	a_2	$P^2/(a_1+a_2)^3$	$4\pi^2/G(m_1+m_2)$
29.9867	1.7857	1.7857	19.7388	19.7392
29.0200	2.6667	1.3334	13.1582	13.1595
0.0628	0.9999	0.0001	0.0039	0.0039
0.7019	4.9970	0.0005	0.0039	0.0039
0.8850	0.9960	0.0020	0.7880	0.7880

Table 2: Shows the values found for the time periods and semimajor axes for each case in checking Kepler's laws. The last two columns show the values found when calculating the lefthand and righthand side of eq(3) for the values in each case.

All five of these test cases used a gravitational constant of $G = 1 \text{ m}^3 \text{ kg}^{-1} \text{ s}^{-2}$, and $\epsilon = 1e-4$, which are the default values used in the code, set as global constants.

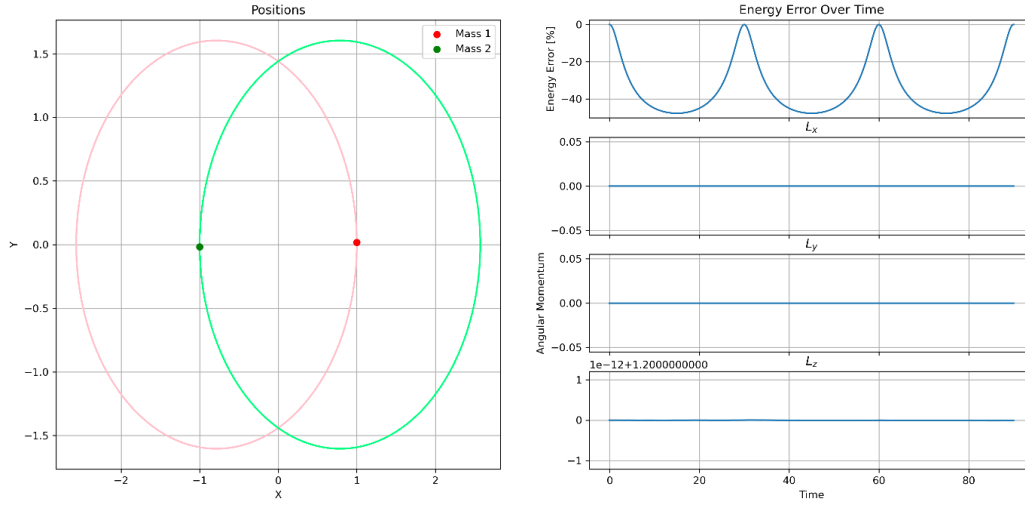


Figure 1: Plot of the first system used to check Kepler's laws. The orbital path is shown on the left and the energy and angular momentum changes are shown on the right.

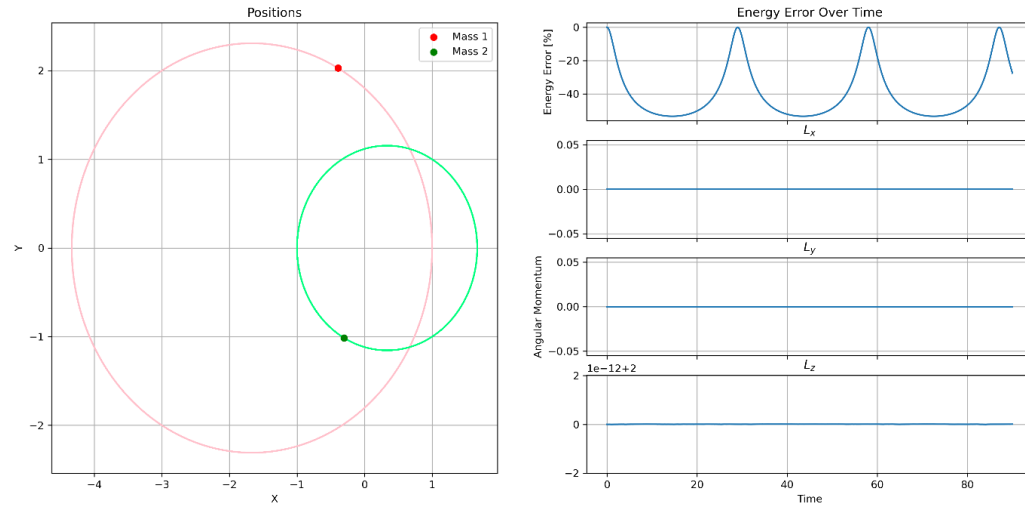


Figure 2: Plot of the second system used to check Kepler's laws. The orbital path is shown on the left and the energy and angular momentum changes are shown on the right.

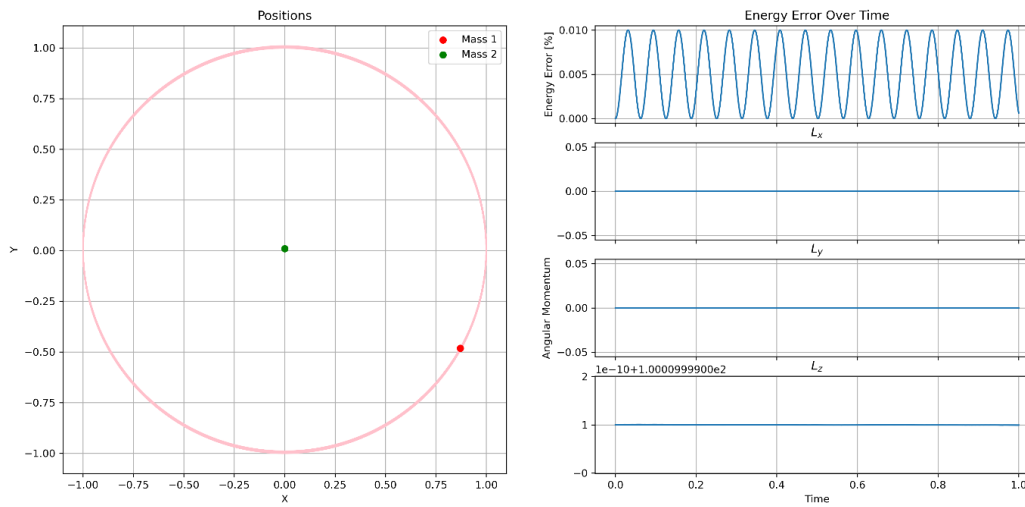


Figure 3: Plot of the third system used to check Kepler's laws. The orbital path is shown on the left and the energy and angular momentum changes are shown on the right.

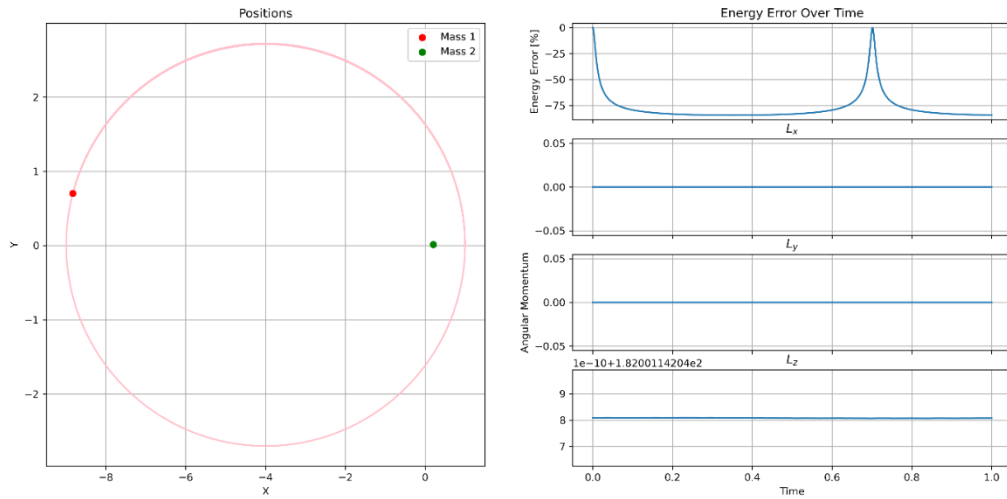


Figure 4: Plot of the fourth system used to check Kepler's laws. The orbital path is shown on the left and the energy and angular momentum changes are shown on the right.

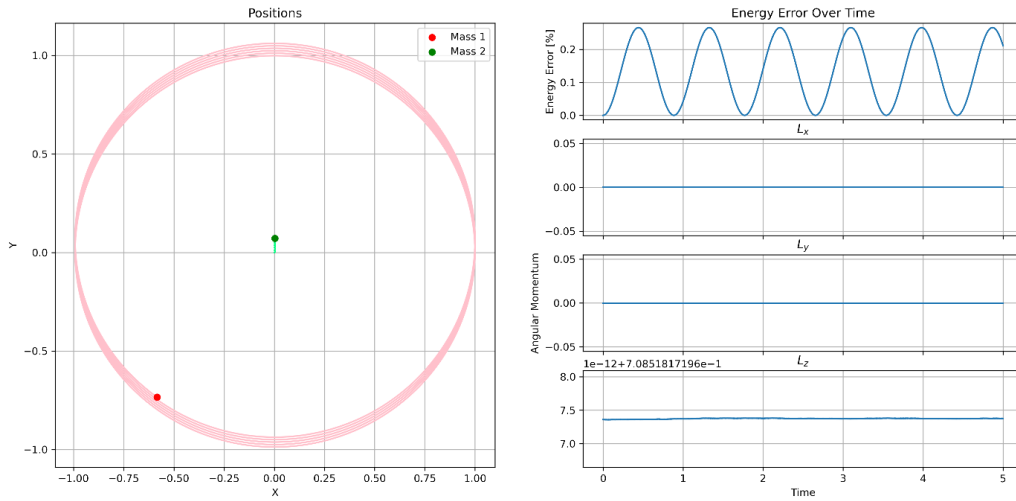


Figure 5: Plot of the fifth system used to check Kepler's laws. The orbital path is shown on the left and the energy and angular momentum changes are shown on the right.

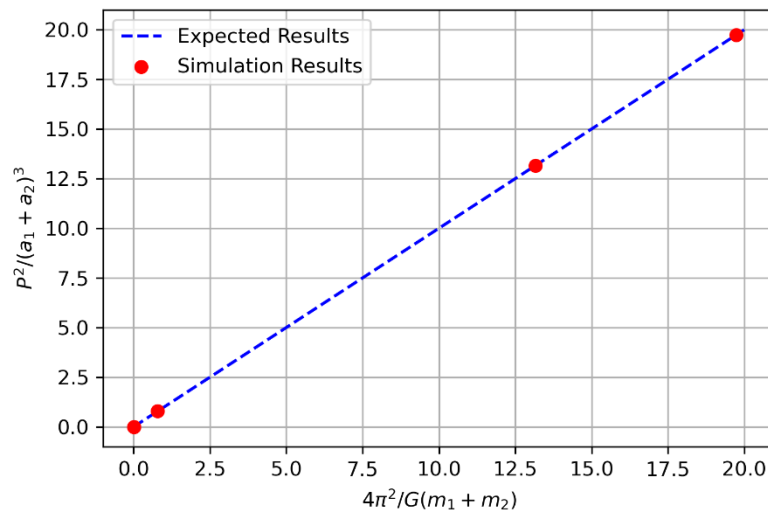


Figure 6: Plot of the last two columns in table 2, showing that the datapoints for checking Kepler's third law match with expectations.

Visually, all the cases plotted show a clear elliptical orbit between the bodies, showing Kepler's first law is obeyed. Some plots have a more circular shape due to the scales of the axes, but all semimajor axes lie on the x-axes, allowing the `find_semimajor_axis()` function to work correctly.

The angular momentum varies on the order of 10^{-10} to 10^{-12} . This variation is small enough that it is sufficient to say that the variation is due to computer accuracy over many iterations and that the angular momentum is correctly conserved, obeying Kepler's second law.

The expected relationship seen in figure 6 is a $y = x$ plot, as both sides of the equation are expected to be the same. This relationship is shown to hold from the datapoints plotted giving good confidence that the code correctly upholds Kepler's third law.

Lastly, the energy for all of the cases oscillates over time but repeatedly falls back to the correct value as is expected with a symplectic integrator. It can also be seen, especially with figure 4, that the energy is best conserved when the bodies are close together.

3 Code Convergence

Whether or not the code correctly models orbits depends on the timestep chosen to run the simulation with, but a smaller timestep will require more computing time so finding the largest timestep possible while still getting the correct orbits is important. Code convergence is shown using three examples in the Jupyter notebook, but the methods used in these examples were the same ones used to generate the plots in Section 4.

The first convergence example uses the figure 8 3-body problem [1]. The timestep ranges from 1s to 0.001s, dividing by 10 each time.

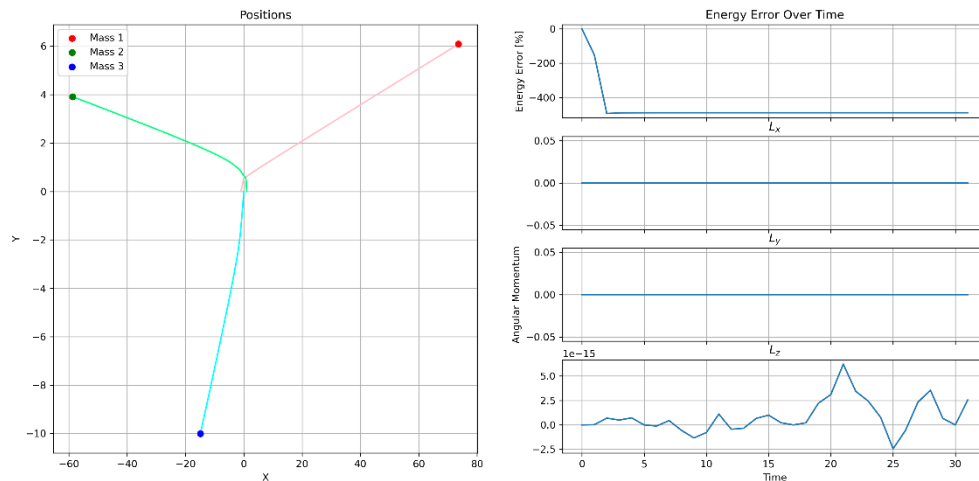


Figure 7: Plot of figure 8 3-body problem, using a timestep, $\delta = 1s$.

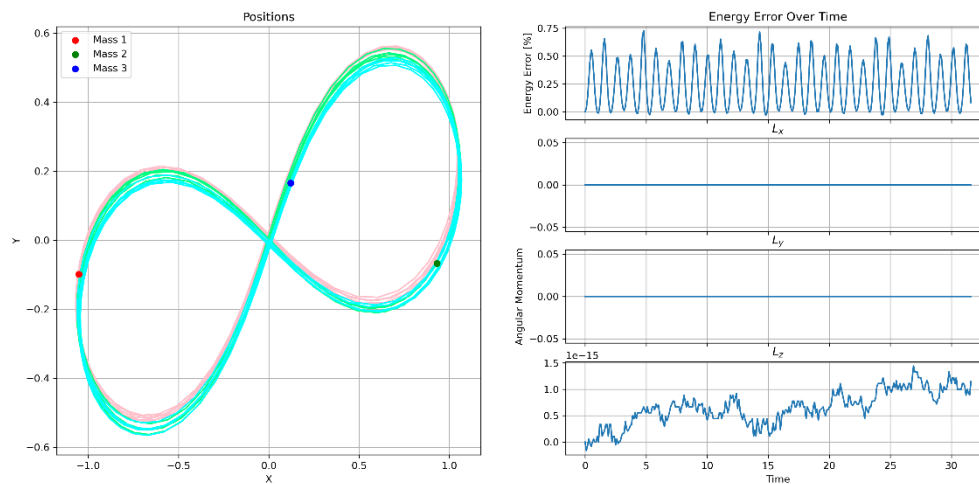


Figure 8: Plot of figure 8 3-body problem, using a timestep, $\delta = 0.1s$.

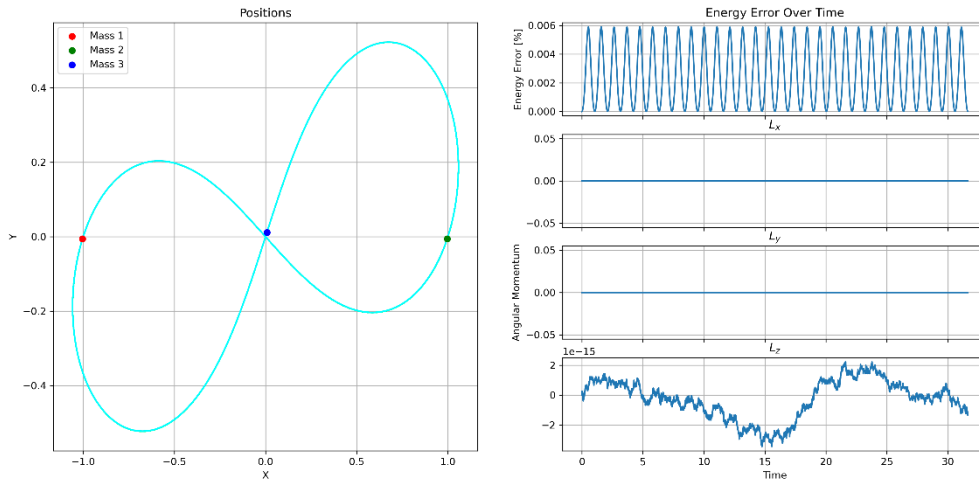


Figure 9: Plot of figure 8 3-body problem, using a timestep, $\delta = 0.01s$.

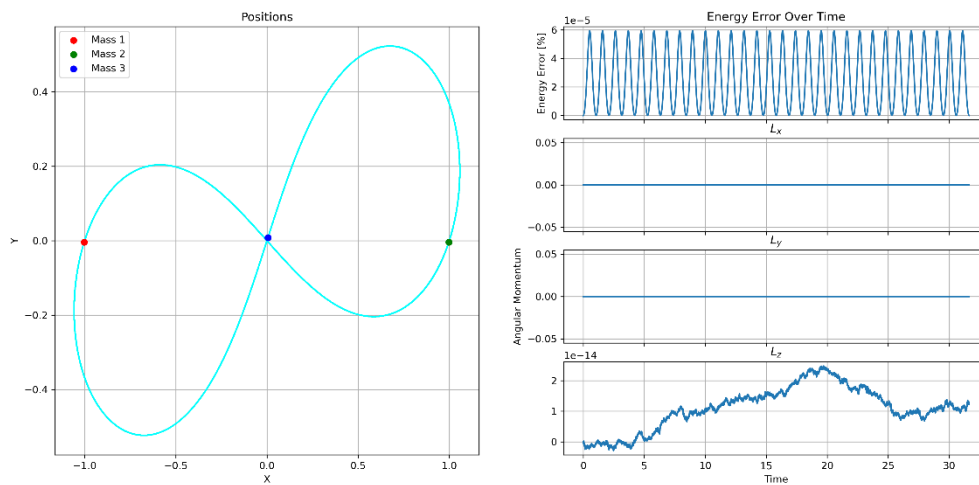


Figure 10: Plot of figure 8 3-body problem, using a timestep, $\delta = 0.001s$.

From figure 7, it is shown that a timestep of 1s is not enough for the model to correctly produce the expected orbits. The larger timestep here means that the bodies move longer distances before their velocities are updated, causing the accelerations between them to be different from the intended result due to their different positions. Angular momentum is still conserved in this case, but the energy settles on a value five times lower than it should be due to the bodies shooting off in different directions instantly.

Figure 8 shows that a timestep of 0.1s is able to roughly produce the correct shape, the energy oscillates back to the correct value repeatedly and the angular momentum is conserved. However, the drifting orbits shows the code hasn't fully converged to the correct solution, meaning a smaller timestep is required.

Figures 9 and 10 both show that the code has fully converged to the correct solution, with all three bodies having overlapping orbits, which is why the figures only show one orbit path. Both simulations conserve energy and angular momentum but the biggest difference is the number of iterations used in each model; 3,163 iterations for figure 9 compared to 31,623 iterations for figure 10. Due to this, a timestep of 0.01s used in figure 9 is better suited for use than a smaller timestep used in figure 10, to save on processing time.

The second convergence example uses the Moth 2 3-body problem, using timesteps between 0.1s and 0.0001s, dividing by 10 each time again.

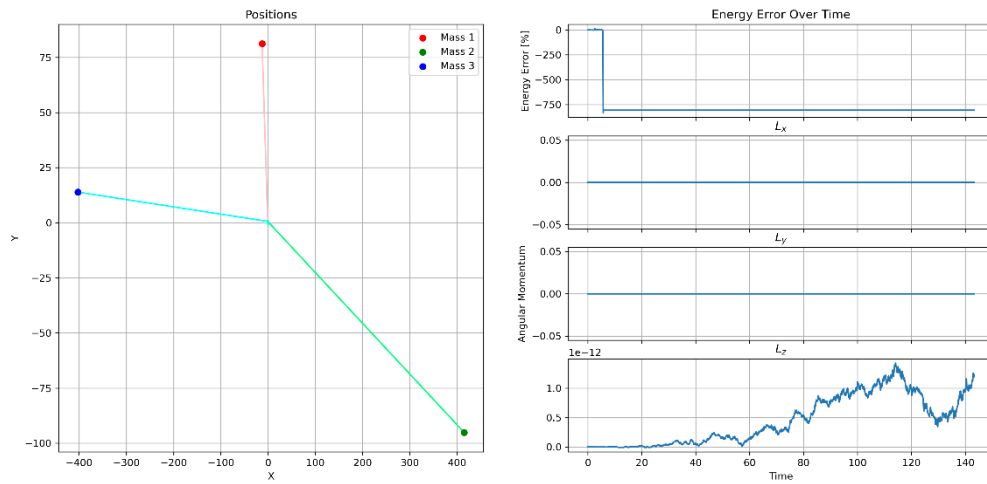


Figure 11: Plot of Moth 2 3-body problem, using a timestep, $\delta = 0.1s$.

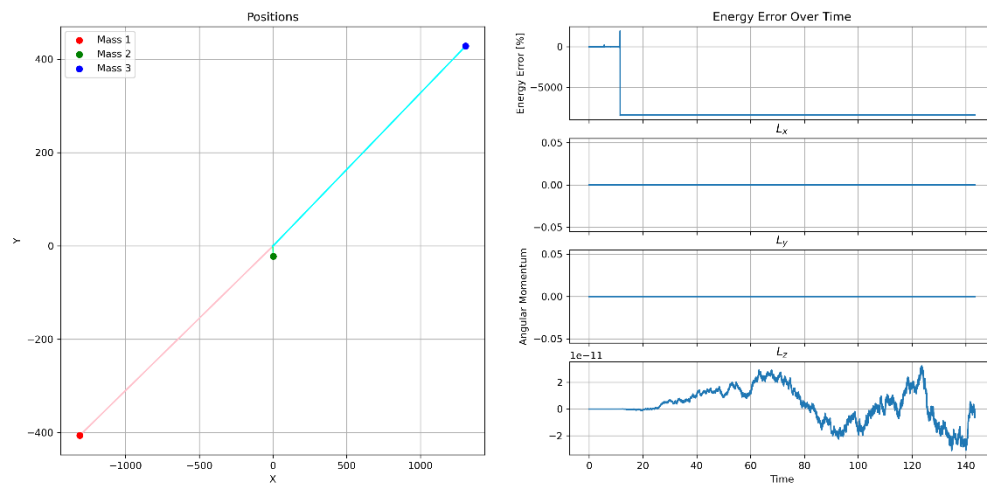


Figure 12: Plot of Moth 2 3-body problem, using a timestep, $\delta = 0.01s$.

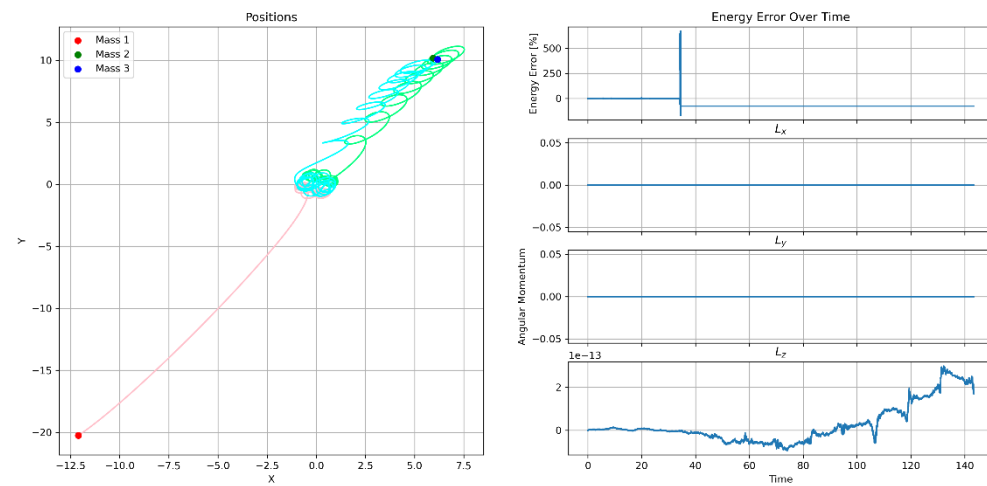


Figure 13: Plot of Moth 2 3-body problem, using a timestep, $\delta = 0.001s$.

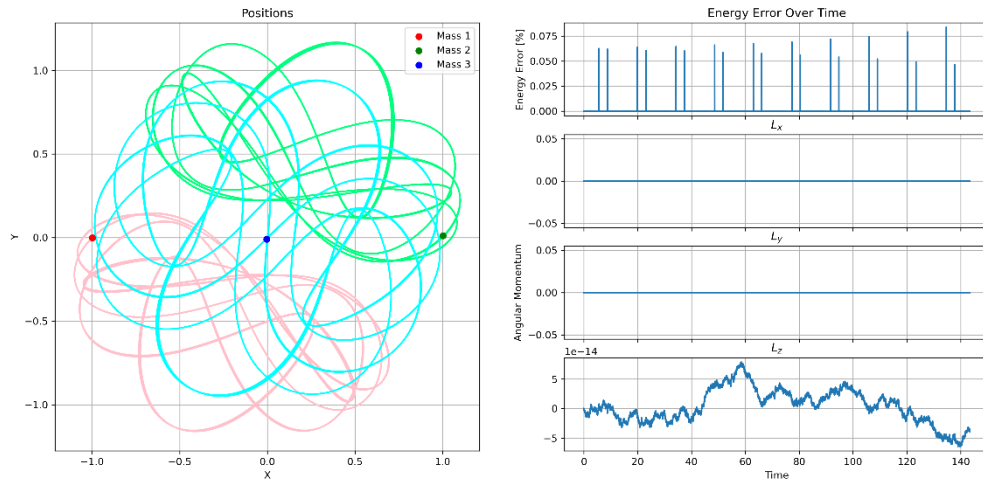


Figure 14: Plot of Moth 2 3-body problem, using a timestep, $\delta = 0.0001s$.

Figures 11-13 all show the code failing to converge correctly to the accepted solution, due to the timestep being too large. But in all of these figures the energy error is stable for some part of the five total orbits, being more stable the smaller the timestep is, and is best seen in figure 13. The reason behind the models falling apart here is because two of the bodies get too close to one another causing there to be a large acceleration that destroys the stable orbit by the next timestep. The epsilon parameter mentioned in the `force_gravity()` function was intended to stop this, but if it is too large it too can break the orbits, so a balance is needed between timestep and epsilon to get a stable orbit in a reasonable number of iterations. The energy is correctly conserved in all these plots up to the point where the orbits are broken, while angular momentum is conserved all the time.

The last code convergence test is to represent the orbits of the planets in the solar system around the Sun, making it a 9-body problem. The data for the positions, velocities and masses of all the bodies involved were found using NASA's Horizons System [2] and their units were converted to be in terms of astronomical units, years and solar masses. All code used to run this is exactly the same, as it is designed to work with N bodies. The standard epsilon value is used, but the gravitational constant is expressed in $[AU^3 M_{sol}^{-1} Yr^{-2}]$.

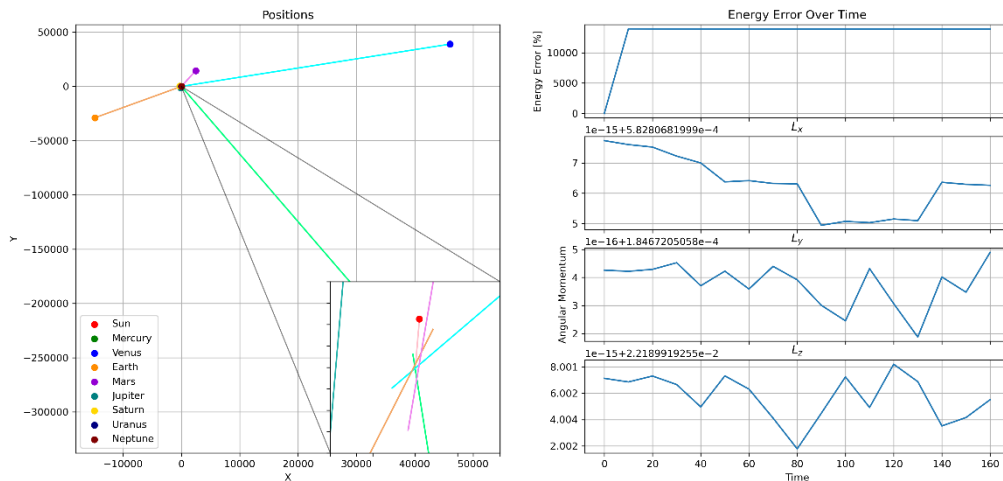


Figure 15: Plot of the solar system planets (9-body problem) using a timestep of 10 Earth years.

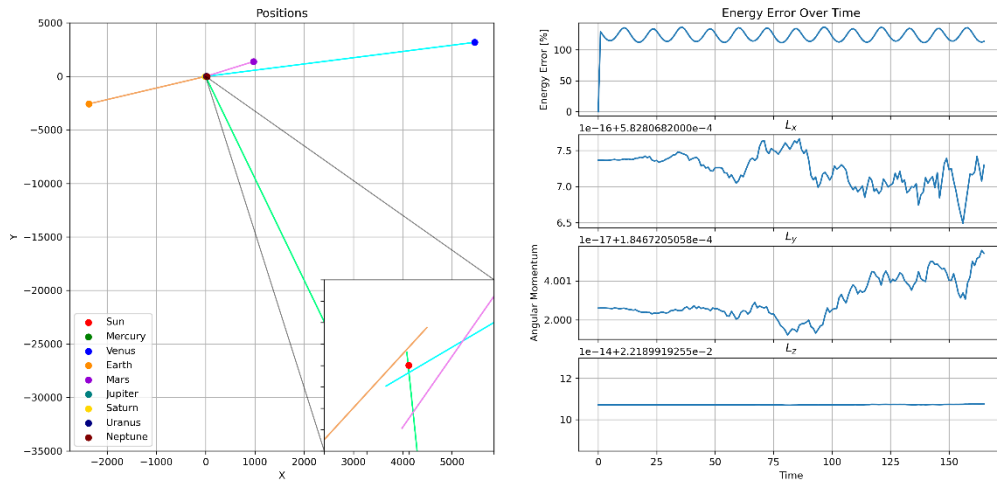


Figure 16: Plot of the solar system planets (9-body problem) using a timestep of 1 Earth year.

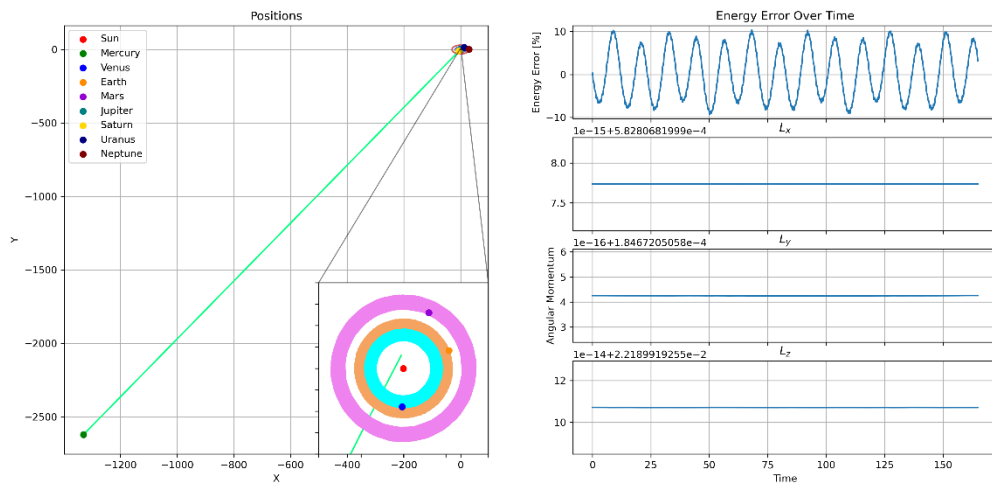


Figure 17: Plot of the solar system planets (9-body problem) using a timestep of 1 Earth month.

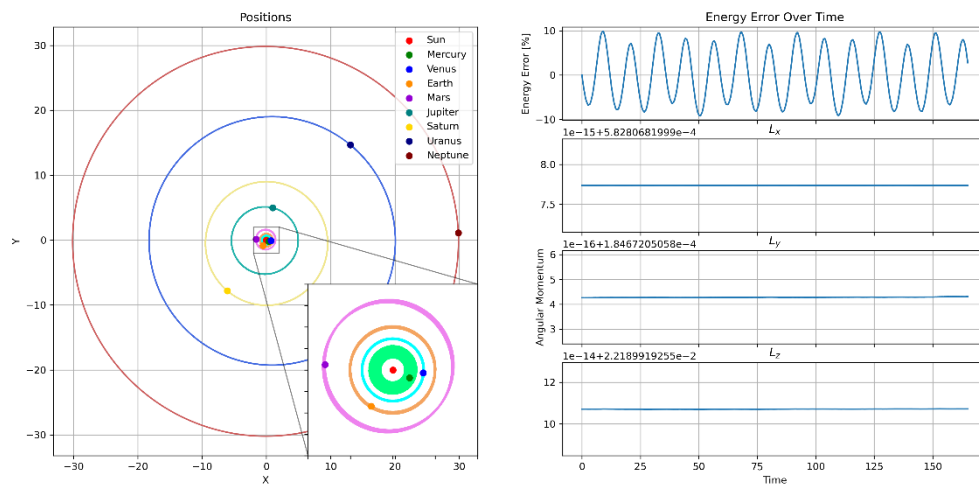


Figure 18: Plot of the solar system planets (9-body problem) using a timestep of 1 Earth week.

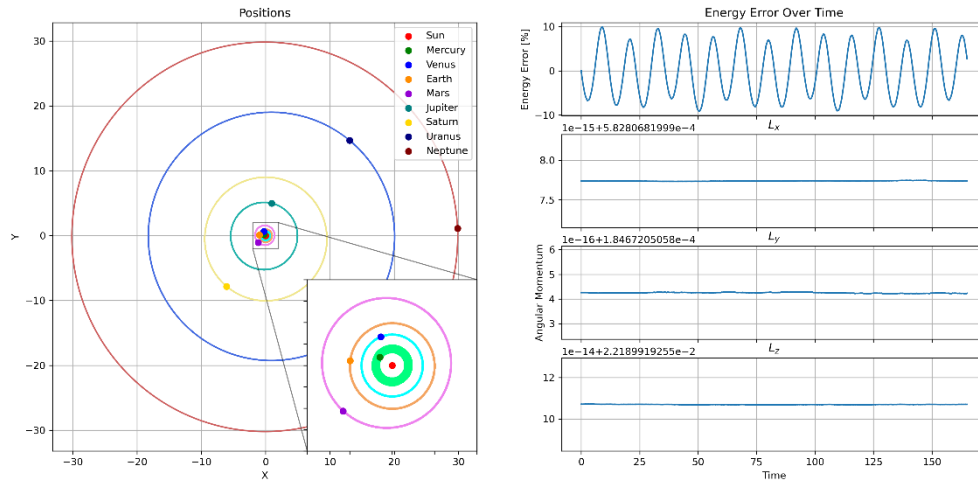


Figure 19: Plot of the solar system planets (9-body problem) using a timestep of 1 Earth Day.

Figures 15 and 16 have timesteps too large to correctly model the orbits of the inner planets, as all the inner planets except Mars would have completed at least one orbit during a timestep of 1 Earth year. The inner planets are seen to shoot off in the direction their first timestep calculated for them, their directions point closer to the Sun indicating the accelerations are correctly being found, just not often enough. However, the timesteps are good enough to model the outer planets orbits. Figure 17 has only Mercury being ejected from a stable orbit, although the remaining inner planets do have varying orbits as seen from the thicker path lines. Figures 18 and 19 show the model has a small enough timestep to converge on a stable orbiting system and both show conserved energy and angular momentum for the systems. Figure 19 has a smaller timestep, so shows a more accurate display for the planet's trajectories, especially for the inner planets, which are more sensitive to a changing timestep.

From these tests it was concluded that the code was capable of converging to stable N-body orbits given a sufficient timestep is chosen.

4 Comparison with Online Examples

A set of 16 different 3-body problems with known solutions, energies and angular momenta was provided for use in this assessment [1]. All 16 were attempted using the code written and 13 of those successfully produced at least one orbit. All systems used the same initial positions for each body: $[-1, 0, 0]$, $[1, 0, 0]$, $[0, 0, 0]$. And the initial velocities are given as: $[V_x, V_y, 0]$, $[V_x, V_y, 0]$, $[-2V_x, -2V_y, 0]$. Each body has a mass of 1, and the gravitational constant $G = 1$, which is the default value in the code. Some systems were not able to be ran for the full five orbits targeted, this is down to the computing time taking too long to feasibly do for this project, whether a system model takes longer to calculate depends on both its period and required timestep.

All systems that were successfully plotted found that the shape of their orbits agreed with the external source, their time periods were the same, and the calculated energies and angular momenta were almost identical showing that the code was working correctly. For most the systems, the energy error oscillated around a 0% error, with occasional spikes in the energy where two bodies are close together, and they have higher kinetic energies as a result, but the models were able to correct the energy. Angular momentum seemed to change randomly, but the deviations were all small enough that it was likely due to computer precision error accumulating over the timesteps.

The three cases that did not manage to converge were for the systems “Butterfly 4”, “Yin-Yang 2a” and “Yin-Yang 2b”. A suitable combination of the timestep and epsilon could not be found without significantly increasing the number of steps in the simulations. Due to all other cases working correctly, it is possible that, given a small enough timestep and long enough computing time, the code

could converge to the correct orbits. But the amount of time required to do this is too long to feasibly run the code for.

Name	V _x	V _y	Period	Expected Energy	Computed Energy	Expected L	Computed L	Timestep
Figure 8	0.347111	0.532728	6.324449	-1.287146	-1.287144	0	10 ⁻¹⁵	0.01
Butterfly 1	0.306893	0.125507	6.235641	-2.170195	-2.170194	0	10 ⁻¹⁵	0.00005
Butterfly 2	0.392955	0.097579	7.003907	-2.008193	-2.008194	0	10 ⁻¹⁵	0.00001
Butterfly 3	0.405916	0.230163	13.86576	-1.846772	-1.846772	0	10 ⁻¹⁵	0.00001
Butterfly 4	0.350112	0.079339	79.47588	-2.113380	N/A	0	N/A	N/A
Bumblebee	0.184279	0.587188	63.53454	-1.363754	-1.363755	0	10 ⁻¹⁵	0.00001*
Dragonfly	0.080584	0.588836	21.27098	-1.440335	-1.440335	0	10 ⁻¹⁵	0.00005
Goggles	0.083300	0.127889	10.46682	-2.430116	-2.430117	0	10 ⁻¹⁵	0.00001*
Moth 1	0.464445	0.396060	14.89391	-1.382281	-1.382282	0	10 ⁻¹⁵	0.001
Moth 2	0.439166	0.452968	28.67028	-1.305861	-1.305860	0	10 ⁻¹⁵	0.0001
Moth 3	0.383444	0.377364	25.84062	-1.631703	-1.631701	0	10 ⁻¹⁵	0.00001*
Yarn	0.559064	0.349192	55.50176	-1.196537	-1.196537	0	10 ⁻¹⁵	0.00001*
Yin-Yang 1a	0.513938	0.304736	17.32837	-1.429011	-1.429011	0	10 ⁻¹⁵	0.00005
Yin-Yang 1b	0.282699	0.327209	10.96256	-1.939048	-1.939047	0	10 ⁻¹⁵	0.00001
Yin-Yang 2a	0.416822	0.330333	55.78983	-1.651418	N/A	0	N/A	N/A
Yin-Yang 2b	0.417343	0.313100	54.20760	-1.683380	N/A	0	N/A	N/A

Table 3: A table containing the parameters for the imported 3-body problems [2] and their names. As well as the computed energies and angular momenta (L) for comparison, and the timesteps required to get a stable orbit/s. The computed energy is the energy calculated using the initial conditions and is the same as a 0% energy error, computed L is the order the deviations are given to in their plots. (*) denotes if the system was modelled for less than 5 orbits with the specific timestep, more details given in each plot's caption.

Plotting the systems given in table 3 gives:

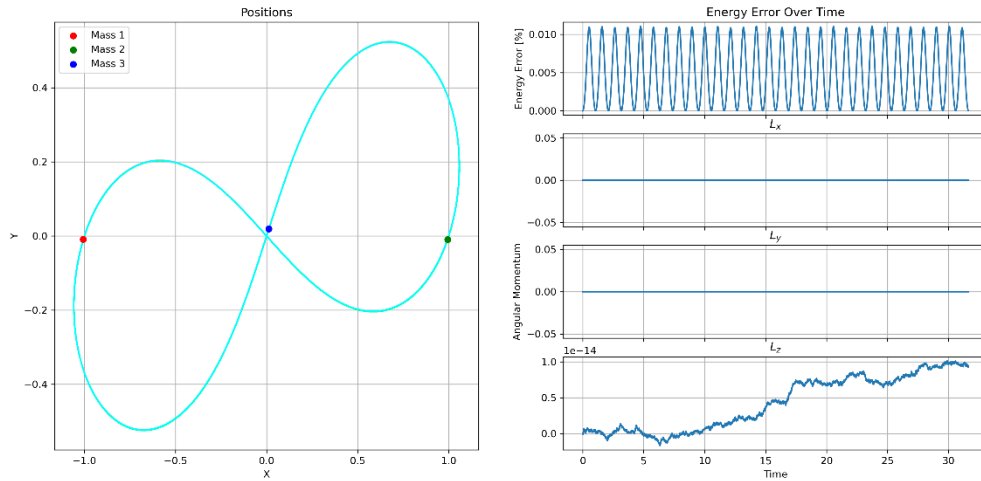


Figure 20: Plot of Figure 8 from table 2, using a timestep, $\delta = 0.01$ and epsilon, $\epsilon = 0.01$. Model ran for 5 orbits.

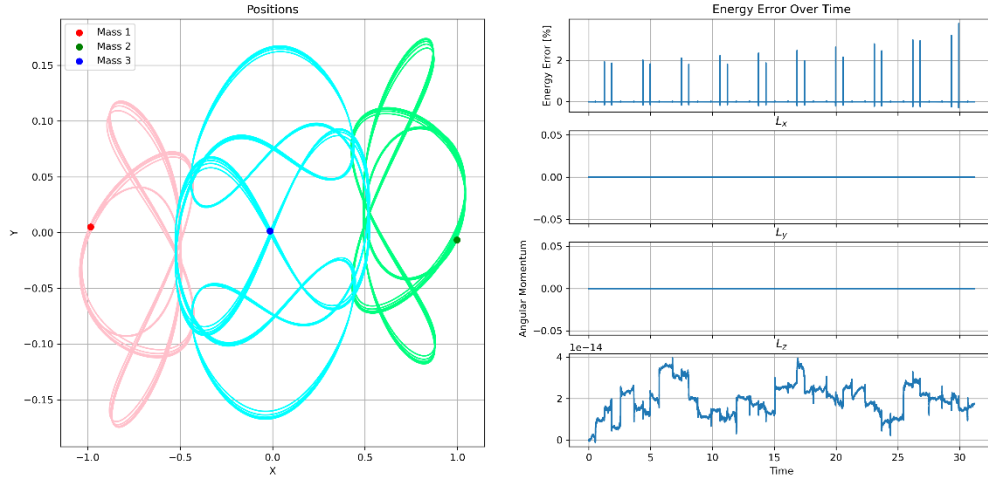


Figure 21: Plot of Butterfly 1 from table 2, using a timestep, $\delta = 0.00005$ and epsilon, $\epsilon = 0.000001$. Model ran for 5 orbits.

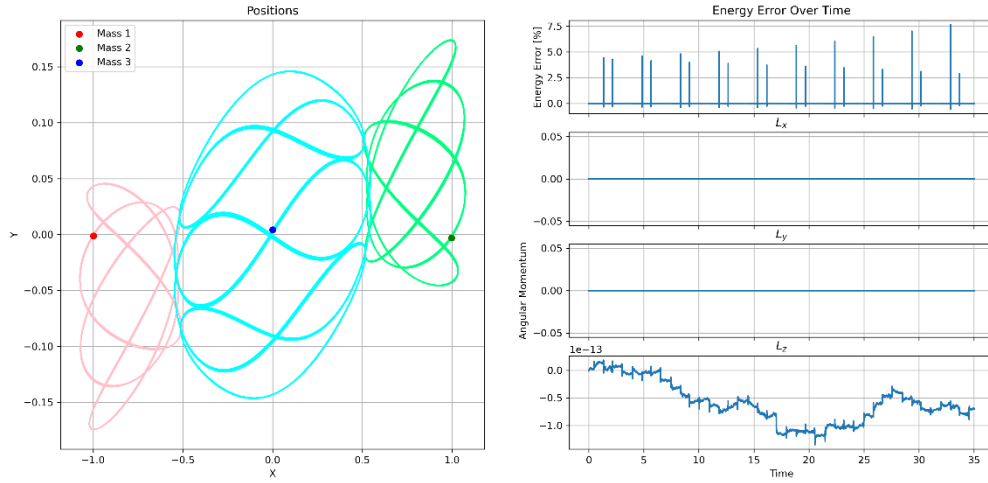


Figure 22: Plot of Butterfly 2 from table 2, using a timestep, $\delta = 0.00001$ and epsilon, $\epsilon = 0.000001$. Model ran for 5 orbits.

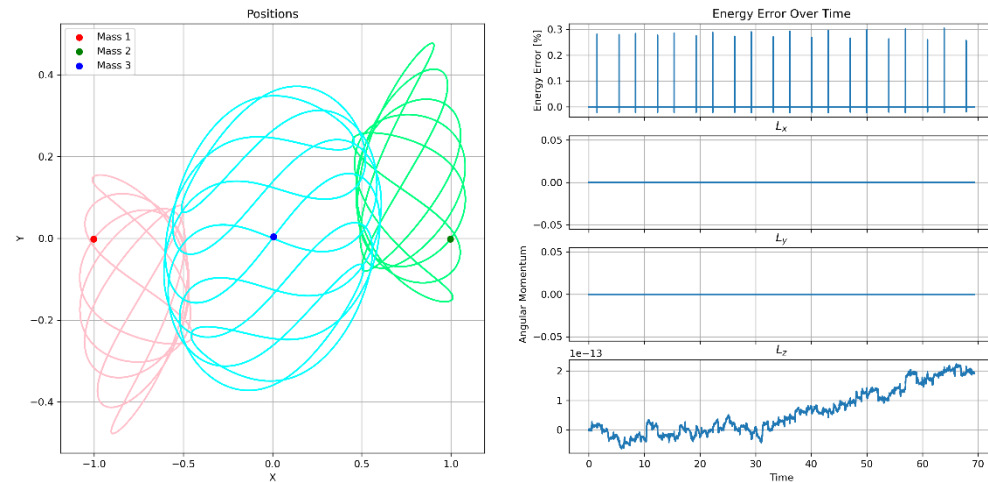


Figure 23: Plot of Butterfly 3 from table 2, using a timestep, $\delta = 0.00001$ and epsilon, $\epsilon = 0.000001$. Model ran for 5 orbits.

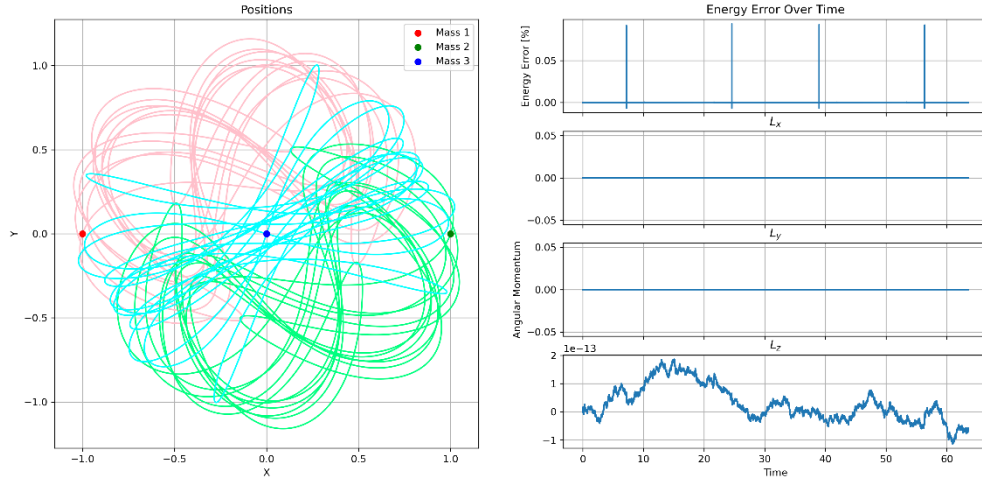


Figure 25: Plot of Bumblebee from table 2, using a timestep, $\delta = 0.00001$ and epsilon, $\epsilon = 0.000001$.
*Model ran for 1 orbit.

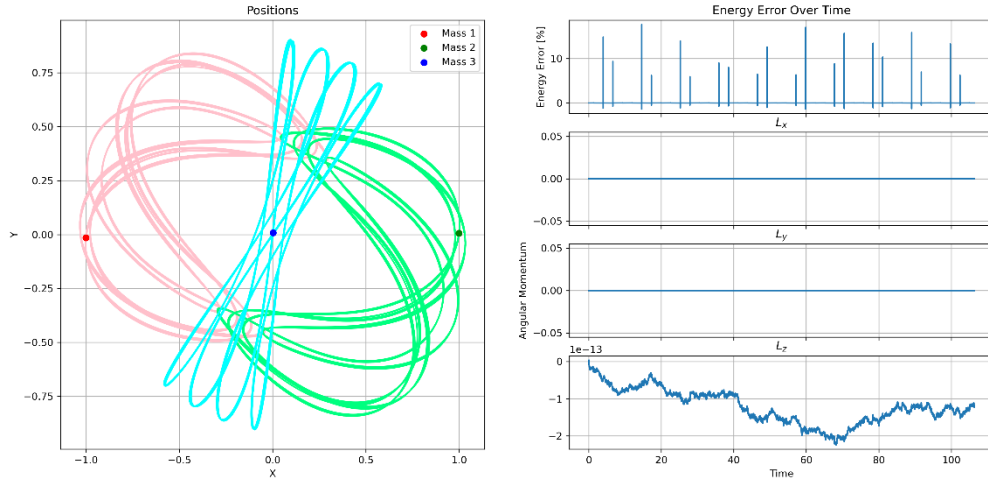


Figure 26: Plot of Dragonfly from table 2, using a timestep, $\delta = 0.00005$ and epsilon, $\epsilon = 0.000001$.
Model ran for 5 orbits.

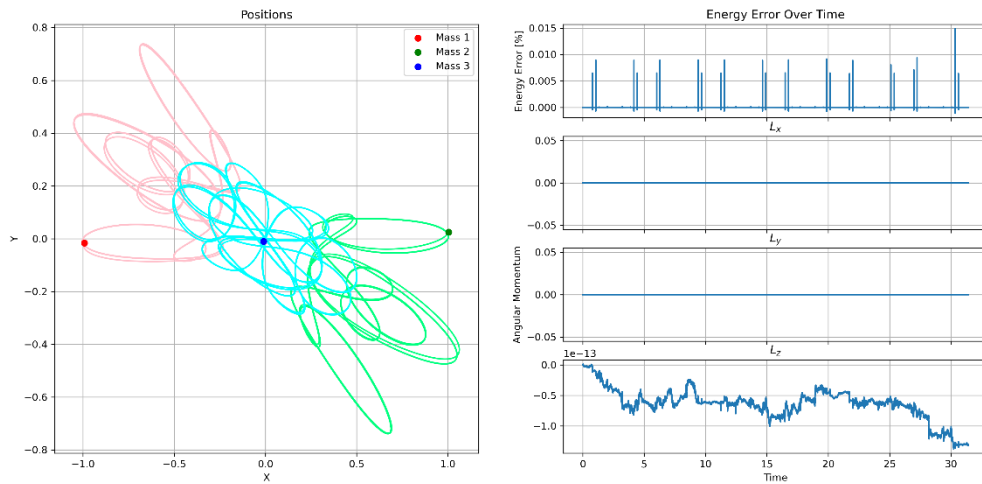


Figure 27: Plot of Goggles from table 2, using a timestep, $\delta = 0.00001$ and epsilon, $\epsilon = 0.000001$.
*Model ran for 3 orbits.

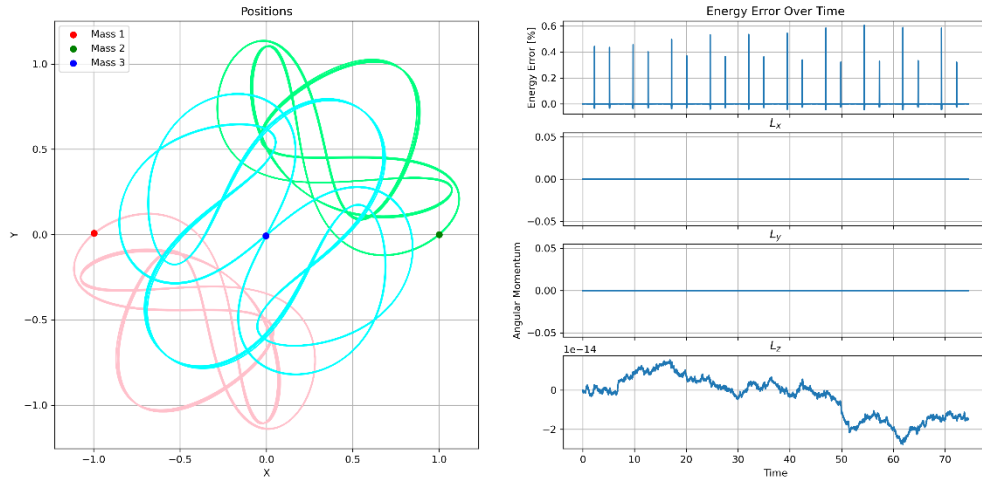


Figure 28: Plot of Moth 1 from table 2, using a timestep, $\delta = 0.001$ and epsilon, $\epsilon = 0.000001$.
Model ran for 5 orbits.

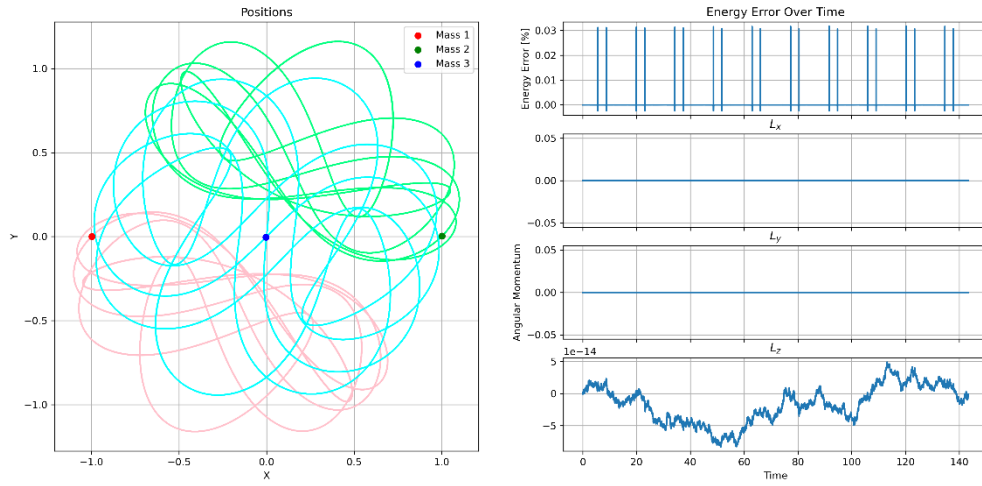


Figure 29: Plot of Moth 2 from table 2, using a timestep, $\delta = 0.0001$ and epsilon, $\epsilon = 0.000001$.
Model ran for 5 orbits.

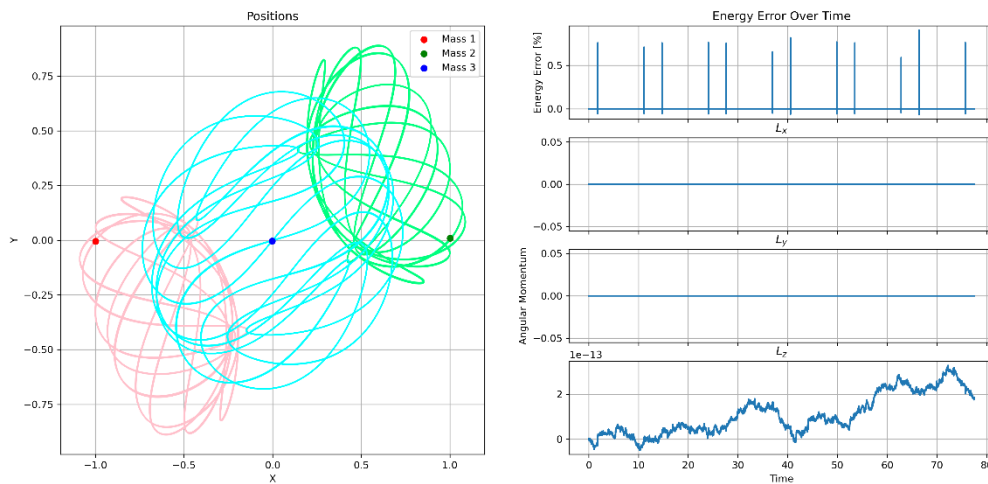


Figure 30: Plot of Moth 3 from table 2, using a timestep, $\delta = 0.00001$ and epsilon, $\epsilon = 0.000001$.
*Model ran for 3 orbits.

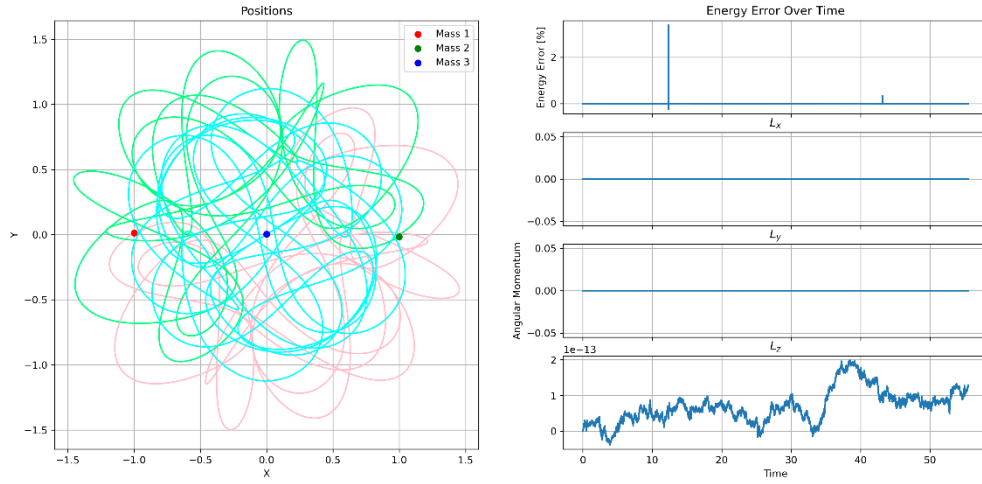


Figure 31: Plot of Yarn from table 2, using a timestep, $\delta = 0.00001$ and epsilon, $\epsilon = 0.000001$.
*Model ran for 1 orbit.

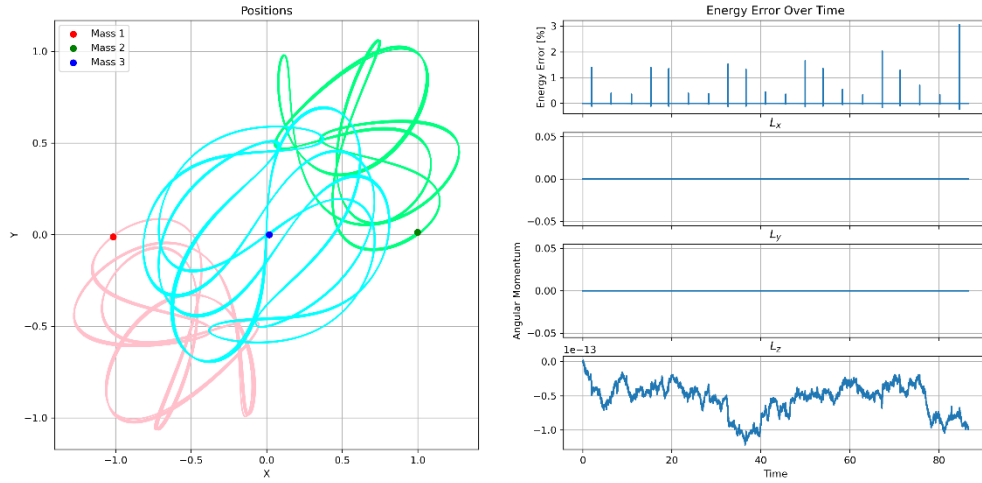


Figure 32: Plot of Yin-Yang 1a from table 2, using a timestep, $\delta = 0.00005$ and epsilon, $\epsilon = 0.000001$.
Model ran for 5 orbits.

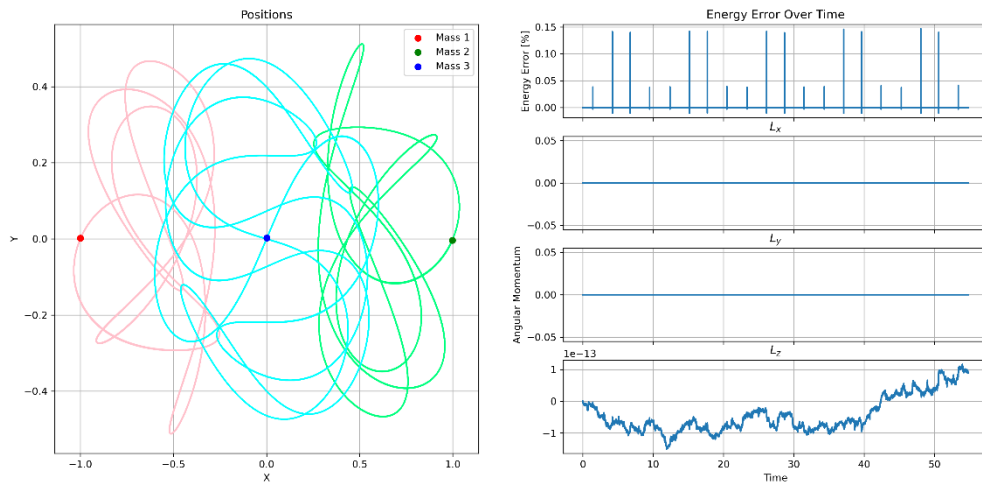


Figure 33: Plot of Yin-Yang 1b from table 2, using a timestep, $\delta = 0.00001$ and epsilon, $\epsilon = 0.000001$.
Model ran for 5 orbits.

5 References

[1] Šuvakov M, Dmitrašinović V. Three classes of Newtonian three-body planar periodic orbits. Physical review letters. 2013 Mar <https://arxiv.org/abs/1303.0181> <http://three-body.ipb.ac.rs/>

[2] NASA. Horizons System [Internet]. 2025 [cited 2025 Dec 2]. Available from: <https://ssd.jpl.nasa.gov/horizons/app.html#/>